Objetivos:

- Construir formulários com Form e TextFormField.
- Ler, alterar e limpar campos com TextEditingController.
- Validar dados e apresentar mensagens de erro.
- Configurar tipos e ações do teclado para facilitar o preenchimento.
- Enviar os dados validados para outra tela.
- Descartar os controladores utilizando dispose().

Indice:

i. Projeto base da aula
ii. Organização do formulário
iii. Leitura e controle dos campos
iv. Validação dos dados
v. Configuração da entrada e envio dos dados

i. Projeto base da aula

Nesta aula, utilizaremos uma aplicação de cadastro de estudante para estudar formulários e validação no Flutter.

O projeto será fornecido completo. Nos próximos capítulos, seus trechos serão utilizados para explicar a organização do formulário, a leitura dos campos, a validação dos dados e o descarte dos controladores.

A aplicação terá duas telas:

- Cadastro**:** preenchimento e validação do nome, e-mail e data de nascimento.
- Resumo**:** apresentação dos dados informados, com possibilidade de voltar ao formulário para alterá-los.

Na tela de cadastro, o usuário deverá preencher todos os campos e pressionar **Exibir resumo**.

Antes de abrir a próxima tela, a aplicação verificará se:

- o nome possui pelo menos três caracteres;
- o e-mail apresenta uma estrutura básica válida;
- a data de nascimento está no formato DD/MM/AAAA, corresponde a uma data válida e não está no futuro.

Se houver algum problema, uma mensagem será apresentada abaixo do campo correspondente. Quando os dados forem válidos, a tela de resumo será aberta.

Ao retornar ao formulário, os dados preenchidos serão mantidos. O botão **Limpar** apagará os campos e as mensagens de validação.

Crie um projeto usando, por exemplo

```
flutter create app –platforms=web,android
```

Substitua o conteúdo de `lib/main.dart` por:

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const CadastroApp());
}

class CadastroApp extends StatelessWidget {
  const CadastroApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Cadastro de estudante',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.indigo,
        ),
        useMaterial3: true,
      ),
      home: const TelaCadastro(),
    );
  }
}

// Representa os dados preenchidos no formulário.
class Estudante {
  final String nome;
  final String email;
  final DateTime dataNascimento;

  const Estudante({
    required this.nome,
    required this.email,
    required this.dataNascimento,
  });
}

// Converte uma data no formato DD/MM/AAAA em DateTime.
// Retorna null quando o texto não representa uma data válida.
DateTime? converterData(String texto) {
  final formato = RegExp(r'^\d{2}/\d{2}/\d{4}$');

  if (!formato.hasMatch(texto)) {
    return null;
  }

  final partes = texto.split('/');
```

```
  final dia = int.parse(partes[0]);
  final mes = int.parse(partes[1]);
  final ano = int.parse(partes[2]);

  if (ano < 1) {
    return null;
  }

  final data = DateTime(ano, mes, dia);

  // DateTime ajusta valores fora dos limites.
  // A comparação permite rejeitar datas como 31/02/2000.
  if (data.day != dia || data.month != mes || data.year != ano) {
    return null;
  }

  return data;
}

// Converte DateTime em um texto no formato DD/MM/AAAA.
String formatarData(DateTime data) {
  final dia = data.day.toString().padLeft(2, '0');
  final mes = data.month.toString().padLeft(2, '0');
  final ano = data.year.toString().padLeft(4, '0');

  return '$dia/$mes/$ano';
}

class TelaCadastro extends StatefulWidget {
  const TelaCadastro({super.key});

  @override
  State<TelaCadastro> createState() => _TelaCadastroState();
}

class _TelaCadastroState extends State<TelaCadastro> {
  final _formKey = GlobalKey<FormState>();

  final _nomeController = TextEditingController();
  final _emailController = TextEditingController();
  final _dataController = TextEditingController();

  String? _validarNome(String? valor) {
    final nome = valor?.trim() ?? '';

    if (nome.isEmpty) {
      return 'Informe o nome.';
    }
```

```
    if (nome.length < 3) {
      return 'O nome deve ter pelo menos três caracteres.';
    }

    return null;
  }

  String? _validarEmail(String? valor) {
    final email = valor?.trim() ?? '';

    if (email.isEmpty) {
      return 'Informe o e-mail.';
    }

    // Verificação simplificada para fins didáticos.
    final formato = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');

    if (!formato.hasMatch(email)) {
      return 'Informe um e-mail válido.';
    }

    return null;
  }

  String? _validarData(String? valor) {
    final texto = valor?.trim() ?? '';

    if (texto.isEmpty) {
      return 'Informe a data de nascimento.';
    }

    final data = converterData(texto);

    if (data == null) {
      return 'Informe uma data válida no formato DD/MM/AAAA.';
    }

    final agora = DateTime.now();
    final hoje = DateTime(agora.year, agora.month, agora.day);

    if (data.isAfter(hoje)) {
      return 'A data de nascimento não pode estar no futuro.';
    }

    return null;
  }

  void _exibirResumo() {
    final formularioValido = _formKey.currentState!.validate();
```

```
    if (!formularioValido) {
      return;
    }

    final estudante = Estudante(
      nome: _nomeController.text.trim(),
      email: _emailController.text.trim(),
      dataNascimento: converterData(_dataController.text.trim())!,
    );

    FocusScope.of(context).unfocus();

    Navigator.push(
      context,
      MaterialPageRoute<void>(
        builder: (context) => TelaResumo(estudante: estudante),
      ),
    );
  }

  void _limpar() {
    _formKey.currentState!.reset();

    _nomeController.clear();
    _emailController.clear();
    _dataController.clear();

    FocusScope.of(context).unfocus();
  }

  @override
  void dispose() {
    _nomeController.dispose();
    _emailController.dispose();
    _dataController.dispose();

    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Cadastro de estudante'),
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Form(
```

```
key: _formKey,
child: Column(
  crossAxisAlignment: CrossAxisAlignment.stretch,
  children: [
    const Text(
      'Informe os dados do estudante.',
      style: TextStyle(fontSize: 18),
    ),
    const SizedBox(height: 24),
    TextFormField(
      controller: _nomeController,
      decoration: const InputDecoration(
        labelText: 'Nome',
        hintText: 'Ex.: Ana Souza',
        border: OutlineInputBorder(),
      ),
      keyboardType: TextInputType.name,
      textCapitalization: TextCapitalization.words,
      textInputAction: TextInputAction.next,
      validator: _validarNome,
    ),
    const SizedBox(height: 16),
    TextFormField(
      controller: _emailController,
      decoration: const InputDecoration(
        labelText: 'E-mail',
        hintText: 'Ex.: ana@email.com',
        border: OutlineInputBorder(),
      ),
      keyboardType: TextInputType.emailAddress,
      textInputAction: TextInputAction.next,
      autocorrect: false,
      validator: _validarEmail,
    ),
    const SizedBox(height: 16),
    TextFormField(
      controller: _dataController,
      decoration: const InputDecoration(
        labelText: 'Data de nascimento',
        hintText: 'DD/MM/AAAA',
        border: OutlineInputBorder(),
      ),
      keyboardType: TextInputType.datetime,
      textInputAction: TextInputAction.done,
      validator: _validarData,
      onFieldSubmitted: (_) => _exibirResumo(),
    ),
    const SizedBox(height: 24),
    ElevatedButton(
```

```
                onPressed: _exibirResumo,
                child: const Text('Exibir resumo'),
              ),
              const SizedBox(height: 8),
              OutlinedButton(
                onPressed: _limpar,
                child: const Text('Limpar'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class TelaResumo extends StatelessWidget {
  final Estudante estudante;

  const TelaResumo({
    super.key,
    required this.estudante,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Resumo do cadastro'),
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const Text(
              'Dados do estudante',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 24),
            Text('Nome: ${estudante.nome}'),
            const SizedBox(height: 12),
            Text('E-mail: ${estudante.email}'),
            const SizedBox(height: 12),
            Text(
              'Data de nascimento: '
```

```
              '${formatarData(estudante.dataNascimento)}',
            ),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Voltar para editar'),
            ),
          ],
        ),
      ),
    );
  }
}
```

ii. Organização do formulário

2.1. O widget Form

O widget `Form` agrupa os campos de um formulário, permitindo executar operações sobre eles em conjunto, como validar ou redefinir seus valores.

No projeto base, ele envolve a coluna que contém os campos e os botões:

```
Form(
  key: _formKey,
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      // Texto de orientação, campos e botões.
    ],
  ),
)
```

Observe as propriedades:

- `key`: recebe `_formKey`, que permite acessar o estado do formulário. Seu funcionamento será explicado no capítulo de validação.
- `child`: define o widget que contém os elementos do formulário. Neste projeto, utilizamos uma `Column`.

O `Form` não determina a disposição visual dos campos. Essa organização é realizada pelos widgets inseridos em seu interior.

2.2. O widget `TextFormField`

O `TextFormField` é um campo de entrada de texto integrado ao formulário. Além de permitir a digitação, ele oferece recursos para validar o conteúdo e apresentar mensagens de erro.

No projeto base, o campo Nome foi definido assim:

```
TextFormField(
  controller: _nomeController,
  decoration: const InputDecoration(
```

```
    labelText: 'Nome',
    hintText: 'Ex.: Ana Souza',
    border: OutlineInputBorder(),
  ),
  keyboardType: TextInputType.name,
  textCapitalization: TextCapitalization.words,
  textInputAction: TextInputAction.next,
  validator: _validarNome,
),
```

Cada propriedade tem uma finalidade:

Propriedade	Finalidade no projeto
`controller`	Permitir a leitura e a alteração do texto do campo.
`decoration`	Configurar rótulo, exemplo e borda.
`keyboardType`	Solicitar um teclado adequado à entrada de nomes.
`textCapitalization`	Solicitar a inicial maiúscula de cada palavra em teclados compatíveis.
`textInputAction`	Configurar a ação do teclado para avançar ao próximo campo.
`validator`	Definir a função que verifica o conteúdo informado.

Os controladores, a validação e as configurações do teclado serão aprofundados nos próximos capítulos.

Diferença entre `TextField` e `TextFormField`: ambos permitem a entrada de texto, mas o `TextFormField` acrescenta a integração com `Form`, incluindo a propriedade `validator`.

2.3. Apresentação dos campos com `InputDecoration`

A propriedade `decoration` recebe um objeto `InputDecoration`, responsável pela apresentação do campo.

Observe o campo E-mail:

```
decoration: const InputDecoration(
  labelText: 'E-mail',
  hintText: 'Ex.: ana@email.com',
  border: OutlineInputBorder(),
),
```

Rótulo: `labelText`

O `labelText` identifica a informação que deve ser preenchida.

Exemplo de preenchimento: `hintText`

O `hintText` apresenta uma orientação enquanto o campo está vazio.

Borda: `border`

A propriedade `border` define a borda do campo.

Nesse caso, `OutlineInputBorder` cria um contorno ao redor da área de entrada.

2.4. Rolagem com `SingleChildScrollView`

Em dispositivos com telas pequenas, ou quando o teclado está aberto, o espaço disponível pode ser insuficiente para apresentar todos os elementos.

Por esse motivo, o formulário foi colocado dentro de um `SingleChildScrollView`:

```
body: SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Form(
    key: _formKey,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        // Elementos do formulário.
      ],
    ),
  ),
),
```

O `SingleChildScrollView` permite rolar o conteúdo quando ele ultrapassa a altura disponível.

A propriedade:

`padding: const EdgeInsets.all(16),`

adiciona um espaço de 16 pixels lógicos em todos os lados, afastando os elementos das bordas da tela.

2.5. Botões do formulário

Ao final do formulário, foram definidos dois botões:

```
ElevatedButton(
  onPressed: _exibirResumo,
  child: const Text('Exibir resumo'),
),
const SizedBox(height: 8),
OutlinedButton(
  onPressed: _limpar,
  child: const Text('Limpar'),
),
```

- Exibir resumo: executa `_exibirResumo()`, que valida os campos e abre a próxima tela quando os dados são válidos.

  ```
  final formularioValido = _formKey.currentState!.validate();
  ```

- Limpar: executa `_limpar()`, que apaga os campos e remove as mensagens de validação.

Na propriedade `onPressed`, passamos a referência do método, sem os parênteses:

`onPressed: _limpar,`

Dessa forma, o Flutter executará o método quando o usuário pressionar o botão.

iii. Leitura e controle dos campos

3.1. O que é um `TextEditingController`?

O `TextEditingController` é um objeto que permite acessar e modificar o texto de um campo de entrada.

No projeto base, foram criados três controladores dentro da classe `_TelaCadastroState`:

```
final _nomeController = TextEditingController();
final _emailController = TextEditingController();
final _dataController = TextEditingController();
```

A declaração com `final` impede que a variável receba outro controlador. Entretanto, o conteúdo do objeto continua podendo ser alterado.

Os controladores são atributos do estado, portanto permanecem disponíveis durante a utilização da tela. Eles não devem ser criados dentro de `build()`, pois esse método pode ser executado várias vezes, criando novos objetos a cada reconstrução.

3.2. Associação do controlador ao campo

A associação é realizada pela propriedade `controller` do `TextFormField`:

```
TextFormField(
  controller: _nomeController,
  decoration: const InputDecoration( ... ),
  validator: _validarNome,
)
```

Quando o usuário digita nesse campo, o texto fica disponível em `_nomeController.text`.

Da mesma forma, uma alteração realizada no texto do controlador é apresentada no campo associado.

3.3. Leitura do texto com a propriedade `text`

A propriedade `text` retorna o conteúdo do campo como uma String.

Por exemplo:

```
nome: _nomeController.text
```

Se o usuário digitar Ana Souza, a variável nome receberá esse texto.

No projeto base, os valores são lidos no método `_exibirResumo()` para criar um objeto `Estudante`:

```
final estudante = Estudante(
  nome: _nomeController.text.trim(),
  email: _emailController.text.trim(),
  dataNascimento: converterData(_dataController.text.trim())!,
);
```

O operador `!` indica que o resultado dessa conversão não será nulo naquele ponto. No projeto, a leitura ocorre depois da validação do formulário, que já verificou se a data é válida.

3.4. Limpeza dos campos com `clear()`

O método `clear()` remove o texto do campo associado.

No projeto base, o método `_limpar()` apaga os três campos:

```
void _limpar() {
  _formKey.currentState!.reset();

  _nomeController.clear();
  _emailController.clear();
  _dataController.clear();

  FocusScope.of(context).unfocus();
}
```

Nesse método:

- `reset()` redefine o estado dos campos do formulário e remove as mensagens de validação;
- `clear()` apaga o texto de cada controlador;
- `unfocus()` retira o foco do campo e normalmente fecha o teclado virtual.

A limpeza do texto e a redefinição da validação são operações distintas. Por isso, o projeto utiliza tanto `reset()` quanto `clear()`.

3.5. Descarte dos controladores com `dispose()`

Os controladores utilizam recursos que devem ser liberados quando o estado da tela for descartado.

Para isso, a classe `_TelaCadastroState` sobrescreve o método `dispose()`:

```
@override
void dispose() {
  _nomeController.dispose();
  _emailController.dispose();
  _dataController.dispose();

  super.dispose();
}
```

O Flutter chama esse método quando o objeto de estado é removido definitivamente da árvore de widgets.

Cada controlador criado pela tela é descartado, e a chamada `super.dispose()` permite que a classe base conclua a liberação dos recursos.

Abrir a tela de resumo não descarta a tela de cadastro. Como `Navigator.push()` mantém a rota anterior na pilha, seus controladores continuam existindo. Por isso, ao voltar, os campos ainda apresentam os dados preenchidos.

iv. Validação dos dados

4.1. Acesso ao estado do formulário

No projeto base, a classe `_TelaCadastroState` possui o seguinte atributo:

```
final _formKey = GlobalKey<FormState>();
```

Essa chave é associada ao widget `Form`:

```
Form(
  key: _formKey,
  child: Column(
    children: [
      // Campos e botões.
    ],
  ),
)
```

O `Form` possui um objeto de estado do tipo `FormState`, responsável por operações como validar e redefinir seus campos.

A chave permite acessar esse objeto por meio de:

```
_formKey.currentState
```

Assim, podemos solicitar a validação do formulário:

```
final formularioValido = _formKey.currentState!.validate();
```

O operador `!` afirma que `currentState` não é nulo naquele momento. No projeto, essa chamada acontece após a construção da tela, quando o `Form` associado à chave está presente na interface.

A chave é criada como atributo do estado para manter sua identidade entre as execuções de `build()`.

4.2. Definição das regras com validator

A propriedade `validator` do `TextFormField` recebe uma função que verifica o conteúdo do campo:

```
TextFormField(
  controller: _nomeController,
  decoration: const InputDecoration( ... ),
  validator: _validarNome,
)
```

Nesse exemplo, `_validarNome` é a função responsável pela validação.

Uma função de validação segue este formato:

```
String? validarCampo(String? valor) {
  //Verificar o conteúdo.

  return null;
}
```

O parâmetro `valor` recebe o texto atual do campo. O retorno determina o resultado.

Retorno	Significado
Uma String com uma mensagem	O conteúdo é inválido. A mensagem será apresentada abaixo do campo.
null	O conteúdo é válido.

4.3. Execução das validações com `validate()`

As funções associadas a `validator` são executadas quando o método `validate()` é chamado.

No projeto base, isso ocorre no início de `_exibirResumo()`:

```
void _exibirResumo() {
  final formularioValido = _formKey.currentState!.validate();
  // ...
}
```

O método `validate()` executa os validadores dos campos do formulário e atualiza as mensagens de erro. Seu retorno é um bool:

- true: todos os campos são válidos;
- false: pelo menos um campo é inválido.

v. Configuração da entrada e envio dos dados

5.1. Tipo de teclado com `keyboardType`

A propriedade `keyboardType` solicita um teclado adequado ao conteúdo esperado pelo campo.

No projeto base, cada campo possui uma configuração:

Campo	Configuração	Finalidade
Nome	`TextInputType.name`	Entrada de nomes.
E-mail	`TextInputType.emailAddress`	Entrada de endereços de e-mail.
Data de nascimento	`TextInputType.datetime`	Entrada de datas e horários.

Em dispositivos móveis, essa configuração pode facilitar o acesso a caracteres como @. A apresentação do teclado depende da plataforma e do teclado instalado.

O tipo de teclado não valida nem restringe, por si só, o conteúdo do campo. Mesmo com um teclado adequado, os dados precisam ser verificados pelo validator.

5.2. Capitalização e correção automática

O campo Nome utiliza:

```
textCapitalization: TextCapitalization.words,
```

Essa configuração solicita a inicial maiúscula de cada palavra em teclados compatíveis, facilitando a digitação de nomes como Ana Souza.

Ela não transforma obrigatoriamente o texto digitado ou colado. O usuário ainda pode informar um nome em letras minúsculas.

No campo E-mail, a correção automática foi desativada:

```
autocorrect: false,
```

Isso evita que o teclado tente corrigir o endereço como se fosse um texto comum.

5.3. Ações do teclado com `textInputAction`

A propriedade `textInputAction` configura a ação apresentada no teclado virtual.

Nos campos Nome e E-mail, utilizamos:

```
textInputAction: TextInputAction.next,
```

A ação indica que o usuário pode avançar ao próximo campo. Na organização do projeto, a sequência é:

1. Nome;
2. E-mail;
3. Data de nascimento.

No último campo, utilizamos:

```
textInputAction: TextInputAction.done,
```

Essa ação indica a conclusão da entrada.

Para executar o envio quando o usuário acionar essa opção, o campo também define:

```
onFieldSubmitted: (_) => _exibirResumo(),
```

O callback recebe o texto do campo. Como o projeto lê os dados pelos controladores, esse parâmetro não é utilizado e foi representado por `_`.

Pressione <enter> em cada um campos e veja que nos campos Nome e E-mail o <enter> faz o foco mover para o campo seguinte e no campo Data de nascimento o <enter> dispara a chamado do método `_exibirResumo()`.

5.4. Retirada do foco com `unfocus()`

Um campo possui foco quando está selecionado para receber a entrada do usuário.

No projeto, o foco é retirado antes de abrir a tela de resumo:

```
FocusScope.of(context).unfocus();
```

Essa chamada remove o foco do campo ativo e normalmente fecha o teclado virtual.

Ela também aparece no método `_limpar()`, evitando que o teclado continue aberto após a limpeza do formulário.

Exercício

Exercício – Cadastro de participante

Desenvolva uma aplicação Flutter com duas telas: uma para preencher os dados de um participante e outra para apresentar o resumo do cadastro, seguindo o funcionamento do projeto base da aula.

1. Tela de cadastro

Organize os seguintes campos em um Form, na ordem apresentada:

Campo	Regras de validação	Tipo de teclado
E-mail	Obrigatório e com estrutura básica válida, como ana@email.com.	TextInputType.emailAddress
Nome	Obrigatório, com pelo menos três caracteres após remover os espaços das extremidades.	TextInputType.name
Telefone	Obrigatório, no formato DD-DDDDD-DDDD.	TextInputType.phone
CPF	Obrigatório, contendo exatamente onze dígitos, sem pontos ou traço. Não é necessário verificar os dígitos verificadores.	TextInputType.number

No campo Telefone, apresente o formato esperado por meio de hintText. O usuário deverá digitar os separadores; não é necessário implementar uma máscara automática.

Configure os campos E-mail, Nome e Telefone com TextInputAction.next, permitindo avançar ao próximo campo pela ação do teclado. No campo CPF, utilize TextInputAction.done e onFieldSubmitted para executar a mesma operação do botão **Exibir dados**.

A tela deverá possuir dois botões:

- **Exibir dados:** validar todos os campos e abrir a segunda tela somente quando os dados forem válidos.
- **Limpar:** apagar os textos, remover as mensagens de validação e retirar o foco do campo ativo.

2. Tela de resumo

Crie uma classe Participante com os atributos imutáveis email, nome, telefone e cpf, todos do tipo String.

Após validar o formulário, crie um objeto dessa classe e envie-o pelo construtor da tela de resumo. Apresente os quatro dados e inclua o botão **Voltar para editar**, que deverá retornar ao formulário mantendo os campos preenchidos.

3. Requisitos de implementação

- Criar uma GlobalKey<FormState> e um TextEditingController para cada campo como atributos do estado.
- Definir uma função de validação para cada campo, retornando uma mensagem em caso de erro e null quando o valor for válido.
- Ler os dados pelos controladores e aplicar trim() antes de criar o objeto.
- Descartar todos os controladores em dispose().
- Utilizar const sempre que possível.